



DDL

Data Definition Language
with SQLite

01 Midterm



▼ Week 9

Topics covered:

- python – lists and dictionaries
- database fundamentals
- relationships and normalization
- DDL and DML with SQLite
- More details and practice test coming soon



02 SQL Architecture



Server Architecture

- Runs as a separate server process
- A server in this context is a separate machine dedicated to hosting the database separate from the client computer
- Multiple clients connect over a network
- Requires configuration (users, ports, authentication)
- Supports concurrent access
- Uses GRANT / REVOKE permissions
- Often paired with web applications

Embedded/Headless

- No separate server process - database file is bundled in application
- Database is a single file (.db)
- Runs inside the application
- Accessed directly via library or CLI
- No built-in user management
- Lightweight and portable



03

DB4S

DB Browser for SQLite (DB4S) is a high quality, visual, open source tool designed for people who want to create, search, and edit SQLite or SQLCipher database files. DB4S gives a familiar spreadsheet-like interface on the database in addition to providing a full SQL query facility. It works with Windows, macOS, and most versions of Linux and Unix.

▼ <https://sqlitebrowser.org/>

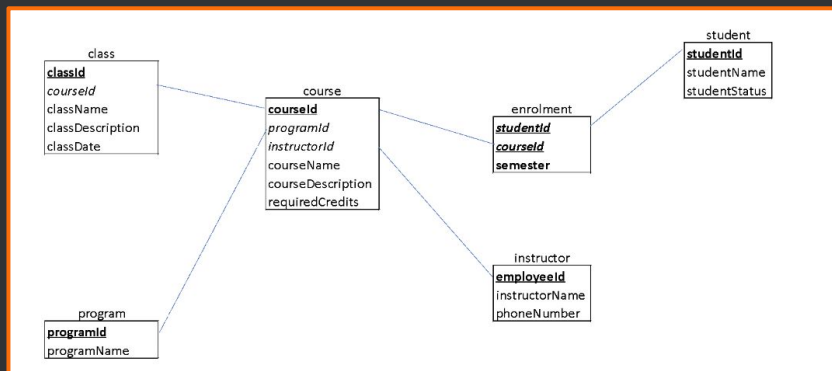


DB Browser for SQLite

light GUI editor for SQLite
databases

04

ERD vs Schema



SQL implements the structure defined in the ERD as schema

```
program(programId, programName)
instructor(employeeId, name, phone)
student(studentId, name, status)
course(courseId, instructorId, programId, name, descriptions,
requiredCredits)
class(classId, courseId, name, description, date)
enrollment(courseId, studentId, semester)
```

05 DDL Commands



CREATE - used to create databases and tables

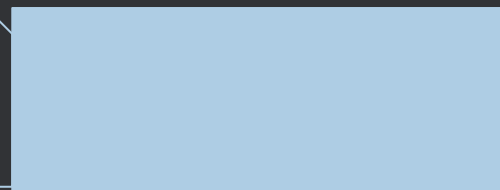
DROP - used to delete databases and tables

ALTER - used to modify structure of tables

In SQLite the database is a file:

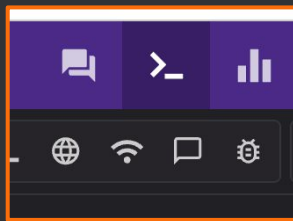
- `sqlite3 nameofdb.db` - connects to an existing database or creates one
- DROP allows you to delete a table, but you must manually delete the .db file
- `ALTER TABLE table_name RENAME TO new_name;`
- `ALTER TABLE table_name ADD COLUMN column_definition;`

▼ 00

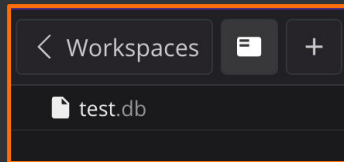


Trying it out

▶ Working with sqlite database files in edstem



```
[user@sahara ~]$ sqlite3 test.db
SQLite version 3.51.2 2026-01-09 17:27:48
Enter ".help" for usage hints.
sqlite> CREATE TABLE student (
(x1...> studentId INTEGER PRIMARY KEY,
(x1...> name TEXT NOT NULL
(x1...> );
```



```
sqlite> .tables
student
```

Create a new workspace in edstem

Access the terminal:

Create a new .db file in edstem

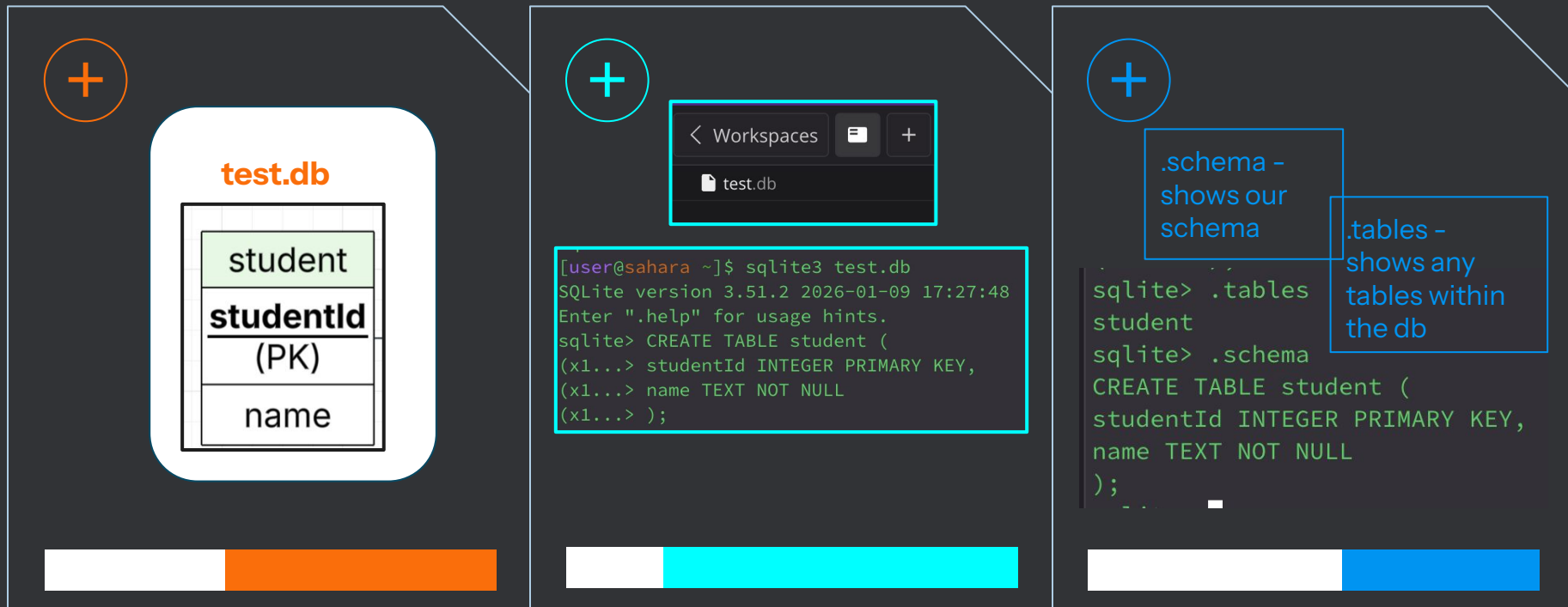
sqlite3 test.db
(we won't see it in the file inspector until we add a table)

so let's add the code above

```
>_ user@sahara:~
[user@sahara ~]$ sqlite3 --version
3.51.2 2026-01-09 17:27:48 b270f8339
[user@sahara ~]$
```

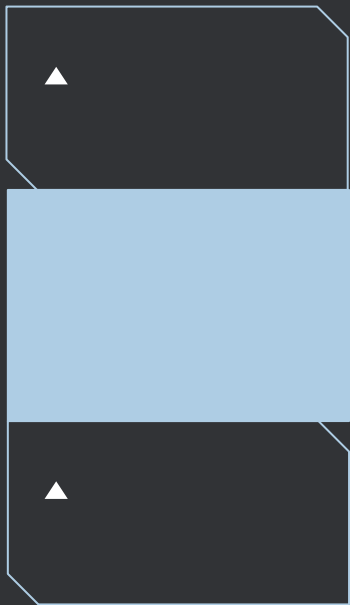
Goal setting

► We just made a database file and created a table within it



06

PERSISTENCE



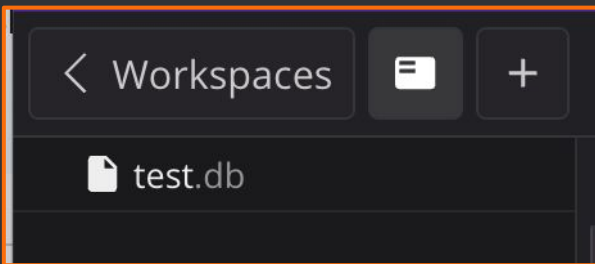
.quit

When we quit the program the .db file still exists. We can even exit the repo and the .db file will still exist when we return. Theoretically, another program can access it.

This is different from the Python exercises with lists and dictionaries we did at the beginning of the semester. There, the data existed only during local runtime.

SQL and SQLite store data persistently on disk.

```
sqlite> .quit  
[user@sahara ~]$
```



07

CREATE TABLE



▲ 00

***This syntax example
is specific to SQLite**

```
sqlite> PRAGMA foreign_keys = ON;
```

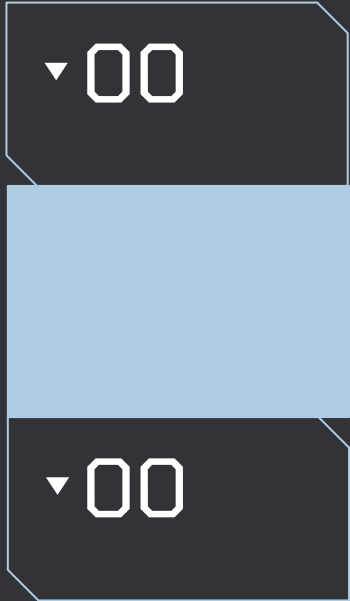
- Foreign key enforcement is disabled by default in SQLite, so we enable it

```
sqlite> CREATE TABLE tbl_name (  
(x1...> field1 INTEGER PRIMARY KEY,  
(x1...> field2 TEXT NOT NULL,  
(x1...> FOREIGN KEY (field2) REFERENCES to_tbl(toField)  
(x1...> );
```

- CREATE TABLE is the command, "tbl_name" is the name of the table we want to create
- field1 is an attribute (column) name INTEGER is the data type and PRIMARY KEY is its constraint
- field2 is an attribute (column) name TEXT is the data type and NOT NULL a constraint
- This defines a foreign key relationship to a PRIMARY KEY or UNIQUE column in another table

08

DATA TYPES (*SQLite type affinity model)



SQLite supports five storage classes:

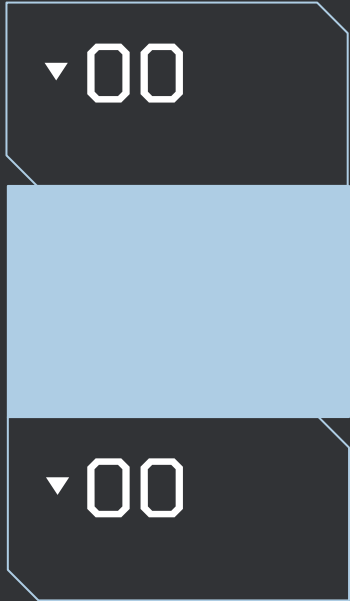
- **INTEGER** – whole numbers
- **REAL** – floating point numbers
- **TEXT** – strings
- **BLOB** – binary data
- **NULL** – missing value

SQLite uses dynamic typing. The declared type influences storage, but values are not strictly constrained.

SQLite does not enforce:

- **char**(n) length (**INTEGER**)
- **varchar**(n) length (**INTEGER**)
- **decimal**(x,2) precision (**REAL**)
- true **datetime** type (**REAL**)

Those are accepted syntactically but not enforced strictly.



- **Primary Key (PK)**
 - Uniquely identifies each row (implies NOT NULL)
- **Not Null (NN)**
 - Field must have a value
- **Unique (UQ)**
 - Field values must be unique
- **Auto-generated Key**
 - INTEGER PRIMARY KEY auto-generates values
- **Check**
 - Enforces a rule that values must satisfy a condition
 - (e.g., CHECK (age >= 0))
- **Default**
 - Inserts a value automatically if none is provided

10

ALTER TABLE



00

*** SQLite supports only limited ALTER TABLE operations:**

RENAME TABLE:

```
sqlite> ALTER TABLE tbl_name RENAME TO new_name;
```

ADD A COLUMN:

```
sqlite> ALTER TABLE tbl_name ADD COLUMN field3 TEXT;
```

MODIFYING A TABLE:

1. Create a new table with the correct schema
2. Copy data over
3. Drop the old table
4. Rename the new one

```
CREATE TABLE new_tbl (  
    field1 INTEGER PRIMARY KEY,  
    field2 TEXT  
);  
  
INSERT INTO new_tbl SELECT field1, field2 FROM tbl_name;  
  
DROP TABLE tbl_name;  
  
ALTER TABLE new_tbl RENAME TO tbl_name;
```

11

DROP TABLE



▼ 00

DELETES A TABLE

```
sqlite> DROP TABLE tbl_name;
```

▼ 00

► Streaming Service

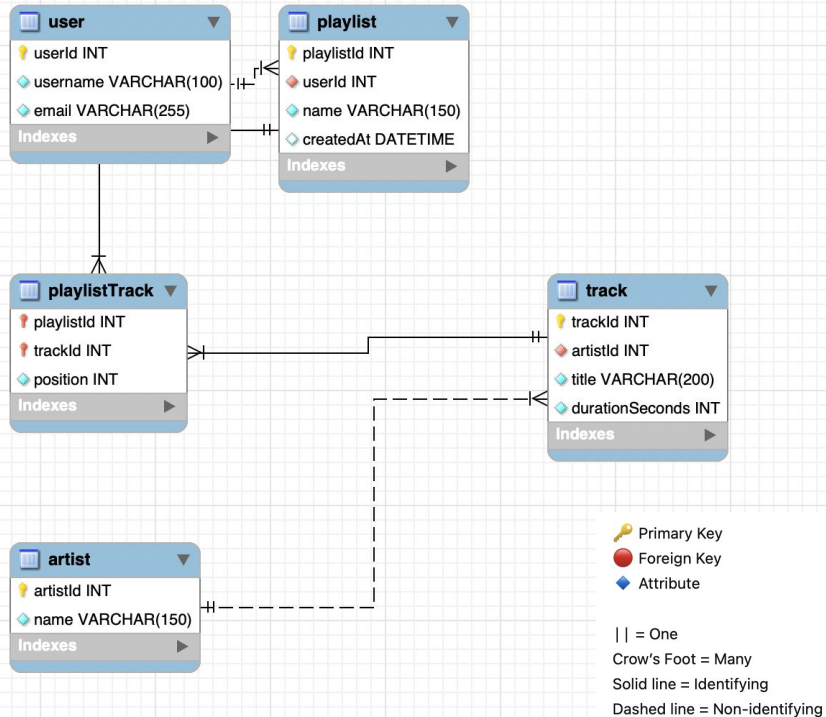
A music streaming app stores information about users, playlists, tracks, and artists. Each user can create multiple playlists. Each playlist contains multiple tracks, and the same track can appear in many playlists (for example, many users can add the same song to different playlists). Each track is performed by one artist. The database also stores the order of tracks inside a playlist so the playlist plays in the correct sequence.

Entities:

- user
- playlist
- artist
- track
- playlistTrack (bridge table)

Example

► Streaming Service - ERD



user (userId, username, email)
playlist (playlistId, userId, name, createdAt)

artist (artistId, name)

track (trackId, artistId, title, durationSeconds)

playlistTrack (playlistId, trackId, position)

Relationships:

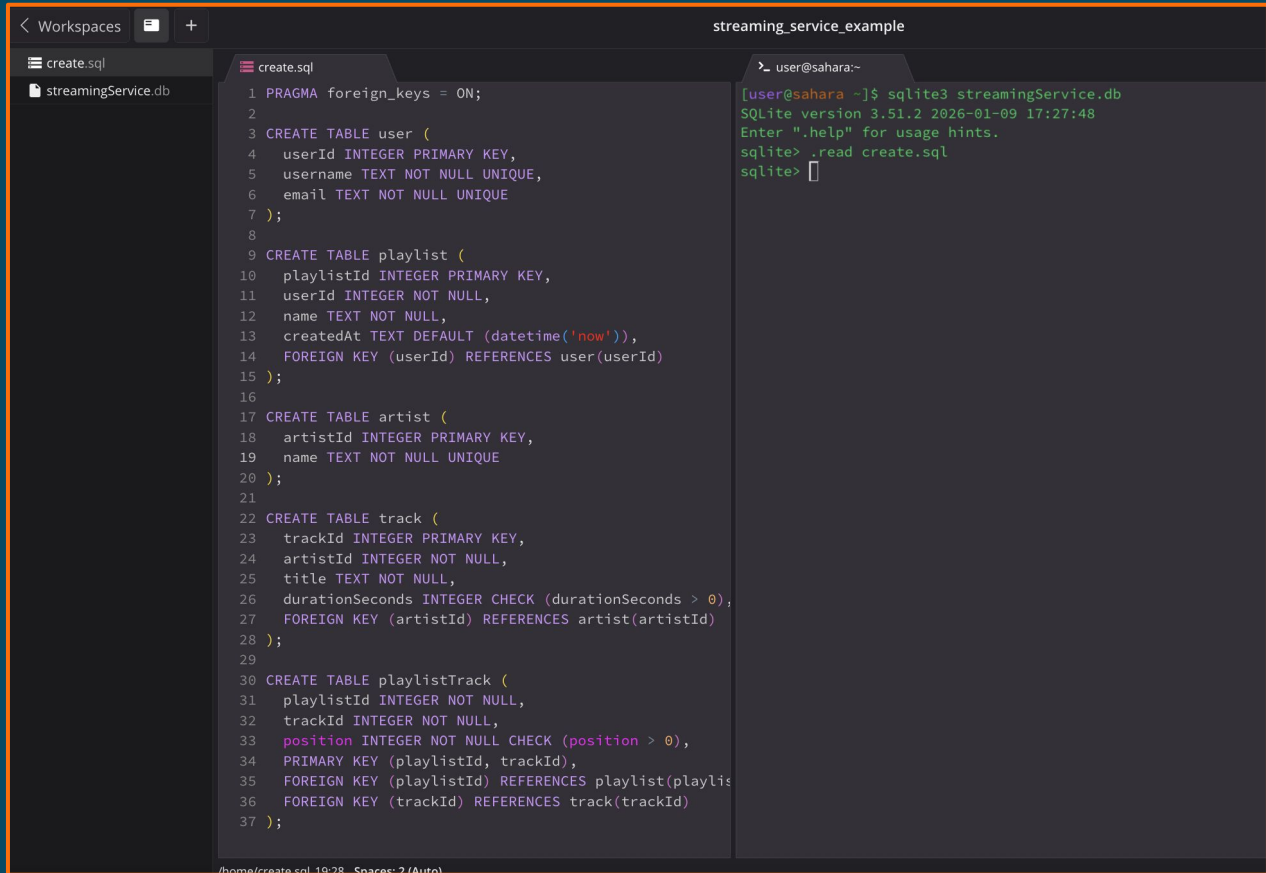
- user 1 → many playlist
- artist 1 → many track
- playlist many ⇔ many track (resolved via playlistTrack)

Example

► Streaming Service - Schema Creation in SQLite

SQL lets us create and enforce the logic designed in the ERD as schema.

*example shown implemented in SQLite



The screenshot shows a code editor with a workspace named 'streaming_service_example'. On the left, a file explorer shows 'create.sql' and 'streamingService.db'. The main editor displays the SQL code for creating the database schema. The code includes tables for users, playlists, artists, tracks, and a join table for playlist tracks, with appropriate primary and foreign key constraints.

```
1 PRAGMA foreign_keys = ON;
2
3 CREATE TABLE user (
4   userId INTEGER PRIMARY KEY,
5   username TEXT NOT NULL UNIQUE,
6   email TEXT NOT NULL UNIQUE
7 );
8
9 CREATE TABLE playlist (
10  playlistId INTEGER PRIMARY KEY,
11  userId INTEGER NOT NULL,
12  name TEXT NOT NULL,
13  createdAt TEXT DEFAULT (datetime('now')),
14  FOREIGN KEY (userId) REFERENCES user(userId)
15 );
16
17 CREATE TABLE artist (
18  artistId INTEGER PRIMARY KEY,
19  name TEXT NOT NULL UNIQUE
20 );
21
22 CREATE TABLE track (
23  trackId INTEGER PRIMARY KEY,
24  artistId INTEGER NOT NULL,
25  title TEXT NOT NULL,
26  durationSeconds INTEGER CHECK (durationSeconds > 0),
27  FOREIGN KEY (artistId) REFERENCES artist(artistId)
28 );
29
30 CREATE TABLE playlistTrack (
31  playlistId INTEGER NOT NULL,
32  trackId INTEGER NOT NULL,
33  position INTEGER NOT NULL CHECK (position > 0),
34  PRIMARY KEY (playlistId, trackId),
35  FOREIGN KEY (playlistId) REFERENCES playlist(playlistId),
36  FOREIGN KEY (trackId) REFERENCES track(trackId)
37 );
```

On the right, a terminal window shows the execution of the SQL script:

```
> user@sahara:~
[user@sahara ~]$ sqlite3 streamingService.db
SQLite version 3.51.2 2026-01-09 17:27:48
Enter ".help" for usage hints.
sqlite> .read create.sql
sqlite>
```

► CREATE A TABLE IN A 'PETS' Database

CHECK W/
.schema

▲ 1 make the .db file

```
[user@sahara ~]$ sqlite3 pets.db
```



● 2 Check slide 9 for some
sqlite field constraint
descriptors



▲ 3 create the table

```
CREATE TABLE tableName (  
  columnName dataType constraint,  
  columnName dataType constraint  
);
```

- petId auto-generates
- name required
- species required
- age ≥ 0
- weight > 0
- adoptionStatus defaults to 'available'
- microchipNumber unique

SCHEMA

EXAMPLE
(won't
actually
make this
pets table)

```
sqlite> CREATE TABLE student (  
  studentId INTEGER PRIMARY KEY,  
  firstName TEXT NOT NULL,  
  lastName TEXT NOT NULL,  
  age INTEGER CHECK (age >= 0),  
  gpa REAL CHECK (gpa >= 0 AND gpa <= 4.0),  
  enrollmentStatus TEXT DEFAULT 'active',  
  email TEXT UNIQUE  
);
```

► INSERT DATA

```
INSERT INTO pet (name, species, age, weight,  
adoptionStatus, microchipNumber)  
VALUES ('Buddy', 'Dog', 3, 45.5, 'available', 'MC1001');
```

```
INSERT INTO pet (name, species, age, weight,  
adoptionStatus, microchipNumber)  
VALUES ('Whiskers', 'Cat', 2, 10.2, 'adopted',  
'MC1002');
```

```
INSERT INTO pet (name, species, age, weight,  
adoptionStatus, microchipNumber)  
VALUES ('Coco', 'Bird', 1, 0.8, 'available', 'MC1003');
```

```
INSERT INTO pet (name, species, age, weight,  
adoptionStatus, microchipNumber)  
VALUES ('Max', 'Dog', 5, 60.0, 'available', 'MC1004');
```

```
INSERT INTO pet (name, species, age, weight)  
VALUES ('Luna', 'Cat', 4, 12.5);
```



```
SELECT * FROM pet;
```

